

Insertion of a new Node into a Red Black Tree

When you insert a new node into a **Red-Black Tree**, it is always initially colored **Red** to avoid violating the black-depth property (which requires every path from the root to a leaf to have the same number of black nodes).

However, inserting a red node can cause a **Red-Black Violation**, specifically a **Red Violation** (Property 4: No two red nodes can be adjacent). Rebalancing and recoloring rules are designed to fix this violation by either pushing the red color up the tree or restructuring the tree locally to restore balance without altering the black-depth property.

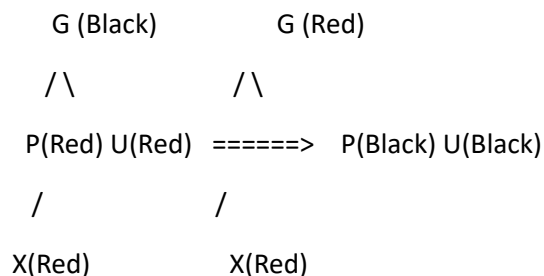
The Base Scenario: Valid Insertion

- **Scenario:** The parent of the new node is **Black**.
- **Action:** Do nothing.
- **Rationale:** Inserting a red node under a black node does not violate any Red-Black properties. The tree remains perfectly valid.

Scenario Group 1: The Uncle is RED (Recoloring Only)

This occurs when the new node (X) has a Red parent (P) and a **Red uncle** (U).

text



Use code with caution.

- **Action:**
 1. Color the Parent (P) and Uncle (U) **Black**.
 2. Color the Grandparent (G) **Red**.
 3. Repeat the violation check with Grandparent (G) as the new node X.
- **Rationale:** Because both P and U are red, we can safely pull the "blackness" down from G. By turning P and U black, we fix the double-red problem between X and P. However, to keep the black-depth consistent for paths passing through G, G must change from black to red. Since G is now red, it might conflict with *its* parent, so the problem is pushed up the tree.

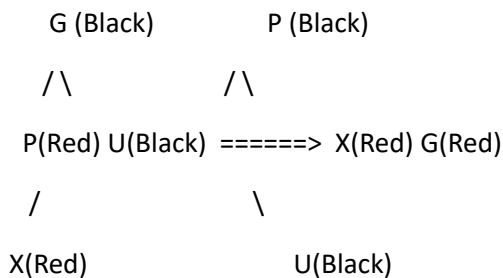
Scenario Group 2: The Uncle is BLACK (Rotations & Recoloring)

When the **uncle (U) is Black** (or non-existent/nil), recoloring alone cannot fix the tree because turning the parent black without changing the uncle would mismatch the black-depths of the two subtrees. We must use structural **rotations**.

Case A: Linear / Straight Line (Left-Left or Right-Right)

- **Scenario:** X is a left child of P, and P is a left child of G (or both are right children).

text



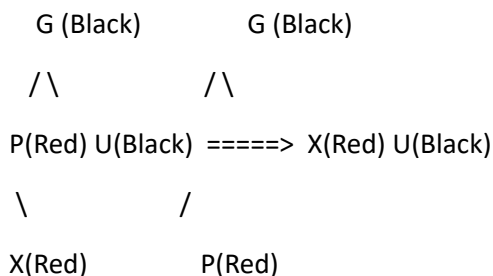
Use code with caution.

- **Action:**
 1. **Rotate** the Grandparent (G) in the opposite direction of the line (e.g., Right-Rotate if it's a Left-Left line).
 2. Swap the colors of the original Parent (P) and Grandparent (G). P becomes **Black**, and G becomes **Red**.
- **Rationale:** The rotation shifts the weight of the tree to perfectly balance the subtrees under P. By making P black, we resolve the double-red violation. Making G red ensures that the total black-depth of all paths passing through this subtree remains exactly the same as it was before the insertion. The conflict is completely resolved locally.

Case B: Triangle / Zig-Zag Line (Left-Right or Right-Left)

- **Scenario:** X is a right child of P, and P is a left child of G (or vice versa).

text



Use code with caution.

- **Action:**
 1. **Rotate** the Parent (P) in the direction of the zig-zag to turn it into a straight line (Case A).

2. Treat P as the new node and apply **Case A rules** (Rotate G and recolor).
- **Rationale:** Red-Black balance rotations require a straight-line ancestral lineage to distribute nodes symmetrically. Rotating around P transforms the "Triangle" configuration into a "Straight Line" (Case A) without changing the black-depth or fixing the red violation yet. It simply sets up the tree structure so that the standard Case A rotation can cleanly rebalance and recolor the tree.

Summary Table

Uncle Color	Node Orientation	Primary Action	Ultimate Goal
Red	Any	Recolor P, U, and G	Pushes the red violation upward toward the root.
Black / Nil	Triangle (Zig-Zag)	Rotate P into a line	Converts the scenario into a straight line.
Black / Nil	Line (Straight)	Rotate G and swap colors of P and G	Restores perfect local balance and ends the fixup.

If the recoloring pushes a Red violation all the way to the **Root**, the root is simply recolored **Black**. This is the only scenario where the overall black-depth of the entire tree increases by 1.